

Нагрузочное тестирование бэкенда

Асинхронный скрипт на Python для нагрузочного тестирования HTTP-бэкенда. Динамически наращивает число параллельных соединений (от 0 до `--connections`), в течение заданного времени шлёт запросы и в реальном времени показывает статистику.



Подробное руководство по использованию — в [USAGE.md](#). Эта документация также доступна в PDF: [README.pdf](#), [USAGE.pdf](#).

Возможности

- Параллельные соединения с плавным наращиванием (ramp-up) от 0 до заданного числа.
- Работа в течение заданного времени (в минутах).
- Обработка ответов: разделение на успешные (2xx/3xx), неуспешные (4xx/5xx) и сетевые ошибки.
- Live-статистика: активные соединения, OK / FAIL / ERR, RPS, средняя задержка.
- Итоговый отчёт с разбивкой по кодам ответов.
- Произвольные заголовки, методы, тело запроса.
- **PDF-отчёт** после прогона: сводка параметров и результатов, графики во времени (соединения, RPS, ответы, задержка), перцентили и разбивка по HTTP-кодам.
- Корректная остановка по Ctrl+C.

Установка (в виртуальном окружении)

Всё ставится в локальный `.venv`, ничего в систему не пишется.

Linux / macOS:

```
bash setup.sh
```

Windows:

```
setup.bat
```

Скрипты сами находят самый свежий установленный Python (3.10+), создают `.venv` и ставят зависимости из `requirements.txt`.

Вручную (любая ОС):

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt # Linux/macOS
# .venv\Scripts\python -m pip install -r requirements.txt # Windows
```

Активация окружения

Чтобы не писать каждый раз путь `.venv/bin/python`, активируйте окружение — тогда команды `python` и `pip` будут указывать на `.venv`:

```
source .venv/bin/activate      # Linux / macOS
# .venv\Scripts\activate.bat   # Windows (cmd)
# .venv\Scripts\Activate.ps1   # Windows (PowerShell)
```

После активации в строке приглашения появится `(.venv)`, и можно запускать просто:

```
python loadtest.py --url http://10.0.8.49:7777/users -c 100 -d 5 -H "x-api-key: ..."
```

Выйти из окружения: `deactivate`.

Дальше в примерах используется явный путь `.venv/bin/python` — он работает **без** активации. Если окружение активировано, можно писать просто `python`.

Запуск

Linux / macOS:

```
.venv/bin/python loadtest.py \
  --url http://10.0.8.49:7777/users \
  --connections 100 \
  --duration 5 \
  --header "x-client-id: 981824de-449d-482a-8578-cba625f7f57c" \
  --header "x-api-key: bee03ffd-3c6f-4e88-8b7b-b4ac673c9cec"
```

Windows:

```
.venv\Scripts\python loadtest.py --url http://10.0.8.49:7777/users -c 100 -d 5 -H "x-api-key: ..."
```

Параметры

Параметр	Кратко	По умолчанию	Описание
<code>--url</code>		— (обязательно)	URL бэкенда
<code>--connections</code>	<code>-c</code>	50	Максимальное число параллельных соединений
<code>--duration</code>	<code>-d</code>	1.0	Время работы в минутах
<code>--ramp</code>		10.0	За сколько секунд нарастить соединения с 0 до максимума
<code>--method</code>	<code>-m</code>	GET	HTTP-метод
<code>--header</code>	<code>-H</code>	—	Заголовок <code>Key: Value</code> (можно указывать многократно)
<code>--body</code>	<code>-b</code>	—	Тело запроса (для POST/PUT)
<code>--timeout</code>	<code>-t</code>	30.0	Таймаут одного запроса, секунд
<code>--report</code>		авто	Путь к PDF-отчёту (по умолчанию <code>loadtest_report_<дата>.pdf</code>)
<code>--no-report</code>		выкл.	Не формировать PDF-отчёт

Пример вывода

Живая строка обновляется раз в секунду:

[42.0s осталось 258.0s] conn: 100 OK: 18432 FAIL: 12 ERR: 0 RPS: 438.8 avg: 221.4ms
--

Поле	Пример	Что означает
прошло времени	42.0s	сколько секунд работает тест
осталось	осталось 258.0s	сколько секунд осталось до конца (по <code>--duration</code>)
conn	conn: 100	активных соединений сейчас (число запущенных воркеров; во время ramp-up растёт от 0 до <code>--connections</code>)
OK	OK: 18432	успешных ответов суммарно с начала теста (HTTP 2xx/3xx)
FAIL	FAIL: 12	неуспешных ответов суммарно (HTTP 4xx/5xx — бэкенд ответил, но ошибкой)
ERR	ERR: 0	сетевых ошибок суммарно (ответ не получен: обрыв, отказ соединения, таймаут)
RPS	RPS: 438.8	запросов в секунду за последнюю секунду (текущая пропускная способность; усреднённый за весь тест RPS — в итоговом отчёте)
avg	avg: 221.4ms	средняя задержка ответа (от отправки запроса до получения ответа)

OK + FAIL + ERR = общее число обработанных запросов. conn — это число параллельных клиентов (воркеров), каждый держит одно keep-alive соединение.

После завершения выводится **итоговый отчёт**: общее число запросов, успешные / неуспешные / ошибки, средний RPS, средняя задержка, доля успешных и разбивка по конкретным HTTP-кодам ответа. Подробнее — в [USAGE.md](#).

PDF-отчёт

После каждого прогона автоматически формируется PDF-отчёт `loadtest_report_<дата>.pdf` (путь меняется флагом `--report`, отключается `--no-report`). Отчёт содержит 4 страницы:

1. **Сводка** — параметры теста и итоговые результаты (включая перцентили задержки p50/p90/p95/p99, min/max).
2. **Динамика во времени** — графики: активные соединения, RPS, накопительные OK/FAIL/ERR, средняя задержка.
3. **Ошибки во времени** — интенсивность ошибок (FAIL/c и ERR/c), доля ошибок за интервал, накопительные ошибки и накопительная доля ошибок.
4. **Распределение** — столбчатая диаграмма HTTP-кодов и гистограмма задержек с отметкой p95.

отчёт в заданный файл

```
.venv/bin/python loadtest.py --url http://10.0.8.49:7777/users -c 100 -d 5 --report report.pdf
```

без PDF-отчёта

```
.venv/bin/python loadtest.py --url http://10.0.8.49:7777/users -c 100 -d 5 --no-report
```

Для PDF нужен `matplotlib` (ставится из `requirements.txt`). Если его нет — тест отработает, а формирование отчёта будет пропущено с предупреждением.

Локальный тестовый сервер

Чтобы проверить скрипт без реального бэкенда, в комплекте есть `testserver.py` — асинхронный (`asyncio`) HTTP-сервер на стандартной библиотеке (без зависимостей). Держит десятки тысяч одновременных соединений (до ~65000) без потоков, умеет имитировать задержку, долю ошибок и проверку API-ключа.

```
# Терминал 1 – поднять сервер (задержка 10мс, 10% ошибок, проверка ключа)
python3 testserver.py --port 7777 --delay 0.01 --error-rate 0.1 --api-key testkey

# Терминал 2 – пустить нагрузку на него
.venv/bin/python loadtest.py --url http://127.0.0.1:7777/users -c 1000 -d 1 -H "x-api-key:
testkey"
```

Параметры сервера: `--host`, `--port`, `--delay` (сек), `--error-rate` (0.0–1.0), `--api-key`, `--backlog`. Полная справка — `python3 testserver.py --help`.

Без ошибок: `--error-rate` задаёт долю ответов с кодом 503 (имитация сбоев). По умолчанию `0.0` — ошибок нет. Чтобы тест шёл «вчистую», просто не указывайте этот флаг.

Много соединений: оба скрипта при старте сами поднимают лимит открытых файлов (дескрипторов) до 200000 — Too many open files больше не возникает. Если среда запрещает повышать лимит, поднимите вручную: `ulimit -n 200000`.

Документация в PDF

`README.md` и `USAGE.md` уже собраны в [README.pdf](#) и [USAGE.pdf](#). Чтобы пересобрать их после правок Markdown, используйте `md2pdf.py`:

```
# зависимости для сборки (один раз)
.venv/bin/pip install -r requirements-docs.txt
brew install weasyprint           # macOS; Ubuntu: sudo apt install weasyprint

# сборка
.venv/bin/python md2pdf.py README.md USAGE.md
```

Скрипт `md2pdf.py` — универсальный конвертер Markdown → PDF (таблицы, подсветка кода, оглавление, кириллица).